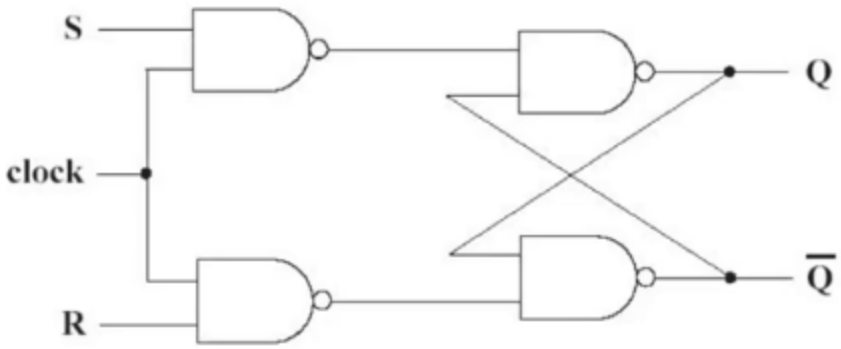
Sequential Circuits - The Clocked SR Flip-Flop

1. Clocked SR Flip-Flop (NAND Implementation)

To control when the SR Latch changes state, we add a clock (or enable) signal. By adding two "steering" NAND gates to the front of a basic active-low NAND latch, we create a Clocked SR Flip-Flop (often called a Gated SR Latch).

- The clock input (`Clk`) acts as a gatekeeper.
- When `Clk = 0` , the steering NAND gates output `1` regardless of S and R. Because the core latch is active-low, a `1` tells it to hold its state.
- When `Clk = 1` , the S and R inputs are inverted and passed through to the core latch, allowing it to function normally.

Circuit Diagram

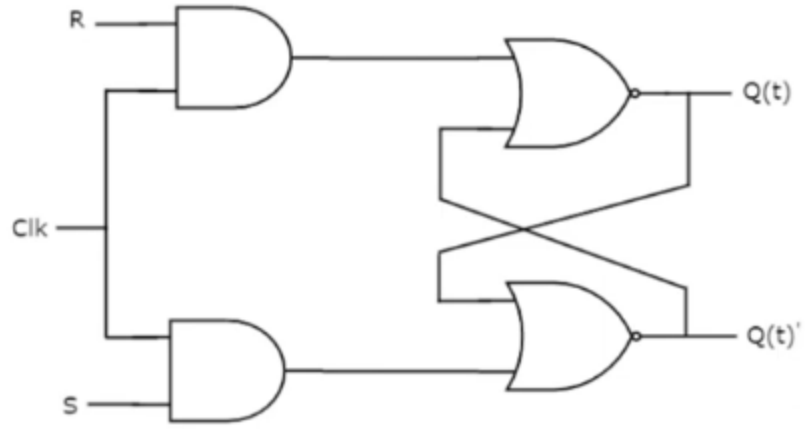


2. Clocked SR Flip-Flop (NOR Implementation)

To build a Clocked SR Flip-Flop around a NOR-gate core, we must use **AND gates** as the steering logic instead of NAND gates.

- The core NOR Latch is naturally an Active-High device (it requires a `1` to change state).
- When `Clk = 0` , the steering AND gates output `0` regardless of the S and R inputs. Supplying `0, 0` to a NOR latch tells it to Hold its state.
- When `Clk = 1` , the S and R inputs pass directly through the AND gates into the core latch without being inverted, triggering the Set or Reset actions normally.

Circuit Diagram



2. Fundamental Flip-Flop Tables

To fully analyze a sequential circuit, we use three specific types of tables. Each serves a different engineering purpose.

A. The Truth Table (Function Table)

The Truth Table defines the overarching behavior of the flip-flop based on the current inputs and the clock.

Clk	S	R	Q_{n+1} (Next State)
0	X	X	Q_n (Hold)
1	0	0	Q_n (Hold)
1	0	1	0 (Reset)
1	1	0	1 (Set)
1	1	1	Invalid

B. The Characteristic Table

The Characteristic Table is used for **analysis**. It explicitly details what the next state (Q_{n+1}) will be for every possible combination of the current state (Q_n) and the current inputs (S, R).

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X (Don't Care/Invalid)
1	1	1	X (Don't Care/Invalid)

C. The Excitation Table

The Excitation Table is used for **design**. If you know what state you are currently in (Q_n) and you know what state you want to go to (Q_{n+1}), this table tells you what inputs (S, R) you must apply to make that transition happen.

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

(Note: X represents a "Don't Care" condition. For example, to transition from $0 \rightarrow 0$, S must be 0, but R can be either 0 or 1, so it is marked as X).

3. The Characteristic Equation

D. K-Map for the Characteristic Equation

To find the equation for the Next State (Q_{n+1}), we plot the outputs from the Characteristic Table onto a 3-variable K-Map based on the inputs S, R, and Q_n.

Note: The Invalid states where $S = 1$ and $R = 1$ are plotted as "Don't Cares" (X). This is a crucial engineering trick, as it allows us to use those cells to make larger groups and simplify the final hardware equation.

$S \backslash RQ_n$	00	01	11	10
0	0	1	0	0
1	1	1	X	X

Grouping the K-Map:

To get the most simplified equation, we circle the largest possible groups of `1`s (incorporating `X`s if they help make the group larger).

1. **Group 1 (The Horizontal Row):** Look at the entire bottom row ($S = 1$). It contains `1, 1, X, X`. We can group all 4 of these cells together. Because this group spans across all possible values of R and Q_n, those variables are eliminated.
   - **Resulting Term:** S
2. **Group 2 (The Vertical Column):** Look at the column where $R = 0$ and $Q_n = 1$ (the `01` column). It contains a `1` in the top row and a `1` in the bottom row. We group these 2 cells vertically. Because this group spans across $S = 0$ and $S = 1$, the S variable is eliminated. The constant variables for this column are $R = 0$ (which is R') and $Q_n = 1$ (which is Q_n).

- **Resulting Term:** $R'Q_n$

The Final Derived Equation:

By ORing (adding) the terms from our two groups, we get the minimized Characteristic Equation for the SR Flip-Flop:

$$Q_{n+1} = S + R'Q_n$$